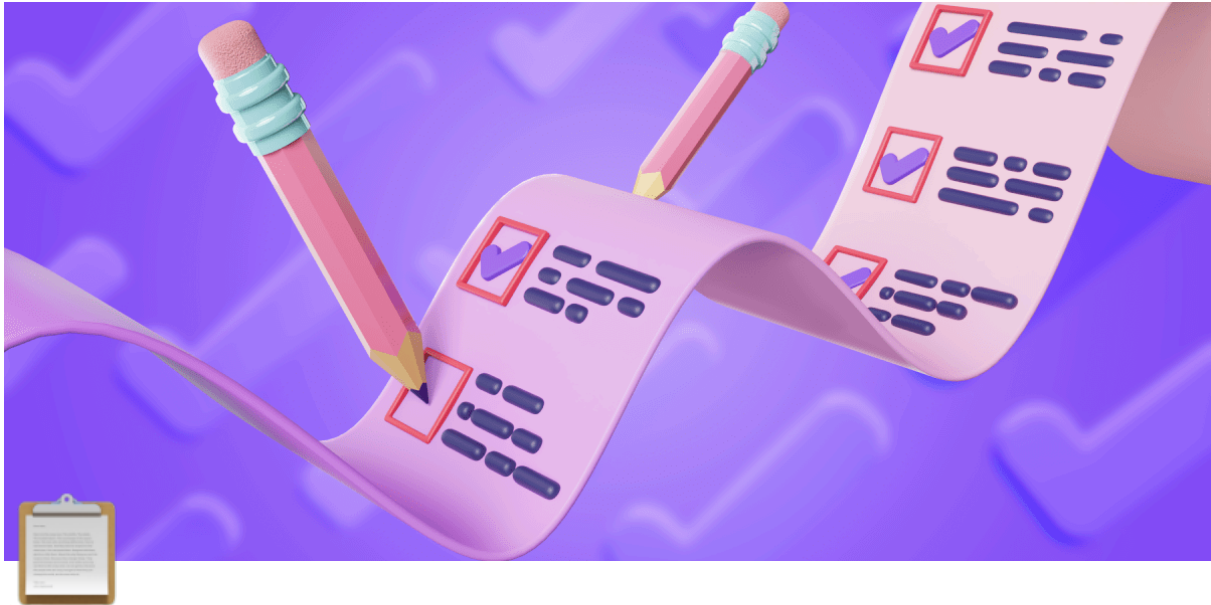




Task Management System

Introduction



The Task Management System is a comprehensive web-based application designed to streamline and enhance project and task management within an organization. This system facilitates collaboration among team members, allowing them to efficiently organize, assign, and track tasks and projects. The application covers user management, project creation, task assignment, category classification, and communication through comments and attachments.

Objective



Task Management System aims to enhance project and task collaboration, improve productivity, and provide valuable insights into project progress and team performance. Its user-friendly interface and robust features make it a valuable tool for organizations seeking efficient task and project management.

Database Schema

▼ ER Diagram

[ER_Diagram.mwb](#)

https://prod-files-secure.s3.us-west-2.amazonaws.com/31282dbc-a6a3-454c-930f-bedcf45363ec/c41505d6-f1b5-4490-8272-fecb939854e9/ER_Diagram.pdf

▼ Database Script File

[create task.sql](#)

Design Guidelines

1. All Rest API requests and response formats must be in JSON.
2. 200, 201, 400, 404, 401, 500, 505 – use the appropriate HTTP status code in your response object.
3. Handle the Rest exceptions globally using the Rest exception handler for all endpoints.
4. Test your Rest Api using Junit and Mockito.

- Participants should not alter database schema under any circumstances except wherever changes are mentioned explicitly.

Rest API Design

All REST API requests and responses follow the JSON format. The project implies standard HTTP status codes (200, 201, 400, 404, 401, 500, 505) appropriately in response objects to convey the outcome of API requests.

Status Codes

HTTP response codes are used to indicate general classes of success and error.

HTTP Status Quote	Description
200	Successfully processed request.
201	Created
202	Accepted
400	Bad Request
401	Unauthorized
403	Forbidden
404	Not Found
500	Internal Server Error

Endpoints

Endpoints

Name	Http Method	Description	Success Response	Error Response
/api/users/post	POST	Create a new user	<pre>{ "code": "POSTSUCCESS", "message": "User added successfully" }</pre>	<pre>{ "code": "ADDFAILS", "message": "User already exist" }</pre>

Aa Name	⚙️ Http Method	≡ Description	≡ Success Response	≡ Error Response
<u>/api/users/all</u>	GET	Get a list of all users	{list of all users}	{ "code": GETFAILS", "message":User list is empty" }
<u>/api/users/{u serId}</u>	GET	Get details of a specific user by ID.	{Object of user }	{ "code": GETFAILS", "message":User doesn't exist exist" }
<u>/api/users/e mail- domain/{dom ain}</u>	GET	Get users with a specific email domain.	{Object of list user }	{ "code": GETFAILS", "message":User doesn't exist exist" }
<u>/api/users/se arch/{name}</u>	GET	Search for users by name	{Object of user }	{ "code": GETFAILS", "message":User doesn't exist exist" }
<u>/api/users/m ost-tasks</u>	GET	users who are assigned the most tasks	{list of Object of user }	{ "code": ADDFAILS", "message":User already exist" }
<u>/api/users/au thenticate</u>	GET	Authenticate a user.	{ "code": "AUTHSUCCESS", "message": "User is valid" }	{ "code": ADDFAILS", "message":User already exist" }
<u>/api/users/co mpleted- tasks</u>	GET	Get users who have completed tasks	{list of Object of user }	{ "code": GETFAILS", "message":User list is empty" }
<u>/api/users/up date/{userId}</u>	PUT	Update user details.	{ "code": "UPDATESUCCE S"	{ "code": UPDTFAILS", }

Aa Name	⚙ Http Method	≡ Description	≡ Success Response	≡ Error Response
			<pre>S", "message": "User updated successfully" }</pre>	<pre>"message":User doesn't exist exist" }</pre>
<u>/api/users/delete/{userId}</u>	DELETE	Delete a user	<pre>{ "code": "DELETESUCCESS", "message": "User deleted successfully" }</pre>	<pre>{ "code": DLTFAILS", "message":User doesn't exist exist" }</pre>
Untitled				
Untitled				
<u>/api/projects/post</u>	POST	Create a new project.	<pre>{ "code": "POSTSUCCESS", "message": "Project added successfully" }</pre>	<pre>{ "code": ADDFAILS", "message":Project already exist" }</pre>
<u>/api/projects/ongoing</u>	GET	Get projects that are currently ongoing.	{list of all projects}	<pre>{ "code": GETALLFAILS", "message":Project list is empty" }</pre>
<u>/api/projects/date-range/{startDate}/{endDate}</u>	GET	Get projects within a specified date range.	{list of all projects}	<pre>{ "code": GETALLFAILS", "message":Project list is empty" }</pre>
<u>/api/projects/user-role/{roleName}</u>	GET	Get projects for users with a specific role.	{list of all projects}	<pre>{ "code": GETALLFAILS", "message":Project list is empty" }</pre>

Name	Http Method	Description	Success Response	Error Response
/api/projects/status/{status}	GET	Get projects by status.	{list of all projects}	{ "code": GETALLFAILS", "message":Project list is empty" }
/api/projects/high-priority-tasks	GET	Get projects with high-priority tasks.	{list of all projects}	{ "code": GETALLFAILS", "message":Project list is empty" }
/api/projects/update/{projectId}	PUT	Update project details.	{ "code": "UPDATESUCCESS", "message": "Project updated successfully" }	{ "code": UPDTFAILS", "message":Project doesn't exist exist" }
/api/projects/delete/{projectId}	DELETE	Delete a project.	{ "code": "DELETESUCCESS", "message": "Project deleted successfully" }	{ "code": DLTFAILS", "message":Project doesn't exist exist" }
/api/tasks/post	POST	Create a new task.	{ "code": "POSTSUCCESS", "message": "Project added successfully" }	{ "code": ADDFAILS", "message":Project already exist" }
/api/tasks/overdue	GET	Get tasks that are overdue.	{List of task}	{ "code": GETALLFAILS", "message":Task list is empty" }
/api/tasks/priority/{priority}	GET	Get tasks by priority and status.	{List of task}	{ "code": GETALLFAILS", "message":Task list is empty" }

Name	Http Method	Description	Success Response	Error Response
<u>/status/{status}</u>				<pre>"message":Task list is empty" }</pre>
<u>/api/tasks/due-soon</u>	GET	Get tasks that are due soon.	{List of task}	<pre>{ "code": GETALLFAILS", "message":Task list is empty" }</pre>
<u>/api/tasks/user/{userId}/status/{status}</u>	GET	Get tasks for a specific user and status	{List of task}	<pre>{ "code": GETALLFAILS", "message":Task list is empty" }</pre>
<u>/api/tasks/category/{categoryId}</u>	GET	Get tasks by category.	{List of task}	<pre>{ "code": GETALLFAILS", "message":Task list is empty" }</pre>
<u>/api/tasks/update/{taskId}</u>	PUT	Update task details.	<pre>{ "code": "UPDATESUCCESS", "message": "Task updated successfully" }</pre>	<pre>{ "code": UPDTFAILS", "message":Task doesn't exist exist" }</pre>
<u>/api/tasks/{taskId}</u>	DELETE	Delete a task.	<pre>{ "code": "DELETESUCCESS", "message": "task deleted successfully" }</pre>	<pre>{ "code": DLTFAILS", "message":Task doesn't exist exist" }</pre>
Untitled				
Untitled				
<u>/api/categories/post</u>	POST	Create a new category.	<pre>{ "code": "POSTSUCCESS", "message": "Category added" }</pre>	<pre>{ "code": ADDFAILS", "message":category }</pre>

Name	Http Method	Description	Success Response	Error Response
			<code>successfully"</code> <code>}</code>	<code>y already exist"</code> <code>}</code>
/api/categories/all	GET	Get a list of all categories.	{List of categories}	<code>{</code> <code>"code":</code> <code>GETALLFAILS",</code> <code>"message":Category</code> <code>ry list is empty"</code> <code>}</code>
/api/categories/{categoryId}	GET	Get details of a specific category by ID.	{Object of Category}	<code>{</code> <code>"code":</code> <code>GETFAILS",</code> <code>"message":category</code> <code>ry doesn't exist"</code> <code>}</code>
/api/categories/task-count	GET	Get categories along with the count of tasks in each category.	<code>{"category":</code> <code>"<count of</code> <code>task>" }</code>	<code>{</code> <code>"code":GETFAILS",</code> <code>"message":Category</code> <code>ry doesn't exist</code> <code>exist"</code> <code>}</code>
/api/categories/update/{categoryId}	PUT	Update category details.	<code>{</code> <code>"code":</code> <code>"UPDATESUCCESS",</code> <code>"message":</code> <code>"Category</code> <code>updated</code> <code>successfully"</code> <code>}</code>	<code>{</code> <code>"code":</code> <code>UPDTFAILS",</code> <code>"message":Category</code> <code>ry doesn't exist"</code> <code>}</code>
/api/categories/{categoryId}	DELETE	Delete a category.	<code>{</code> <code>"code":</code> <code>"DELETESUCCESS</code> <code>",</code> <code>"message":</code> <code>"Category deleted</code> <code>successfully"</code> <code>}</code>	<code>{</code> <code>"code":</code> <code>DLTFAILS",</code> <code>"message":Category</code> <code>ry doesn't exist</code> <code>exist"</code> <code>}</code>
Untitled				
/api/taskcategories/post	POST	Associate a task with one or more categories.	<code>{</code> <code>"code":</code> <code>"POSTSUCCESS",</code>	<code>{</code> <code>"code":</code> <code>ADDFAILS",</code>

Name	Http Method	Description	Success Response	Error Response
			<pre>"message": "Task-category added successfully" }</pre>	<pre>"message":Task- Category already exist" }</pre>
/api/taskcategories/{taskId}	GET	Get all category for a particular task	{List of all category}	<pre>{ "code": GETFAILS", "message":No category found for a particular task" }</pre>
/api/taskcategories/{categoryId}	GET	Get all tasks for a particular task	{List of all task}	<pre>{ "code": GETFAILS", "message":No task found for a particular category" }</pre>
/api/comments/post	POST	Create a new comment.	<pre>{ "code": "POSTSUCCESS", "message": "Comments added successfully" }</pre>	<pre>{ "code": ADDFAILS", "message":Comme nts already exist" }</pre>
/api/comments/all	GET	Get a list of all comments.	{List of comments}	<pre>{ "code": GETALLFAILS", "message":Comme nt list is empty" }</pre>
/api/comments/{commentId}	GET	Get details of a specific comment by ID.	{Object of comment}	<pre>{ "code": GETFAILS", "message":Comme nt doesn't exist" }</pre>
/api/comments/update/{commentId}	PUT	Update comment details.	<pre>{ "code": "UPDATESUCCESS", }</pre>	<pre>{ "code": UPDTFAILS", }</pre>

Aa Name	🔗 Http Method	≡ Description	≡ Success Response	≡ Error Response
			<pre>"message": "Comment updated successfully" }</pre>	<pre>"message":category doesn't exist" }</pre>
/api/comments/delete/{commentId}	DELETE	Delete a comment.	<pre>{ "code": "DELETESUCCESS", "message": "Comment deleted successfully" }</pre>	<pre>{ "code": DLTFAILS", "message":Comments doesn't exist exist" }</pre>
/api/attachments/post	POST	Upload a new attachment.	<pre>{ "code": "POSTSUCCESS", "message": "Attachment added successfully" }</pre>	<pre>{ "code": ADDFAILS", "message":Attach ment already exist" }</pre>
/api/attachments/all	GET	Get a list of all attachments.	{List of object of Attachment}	<pre>{ "code": GETALLFAILS", "message":Attach ment list is empty" }</pre>
/api/attachments/{attachmentId}	GET	Get details of a specific attachment by ID.	{Object of Attachment}	<pre>{ "code": GETFAILS", "message":Attach ment doesn't exist" }</pre>
/api/attachments/update/{attachmentId}	PUT	Update attachment details.	<pre>{ "code": "UPDATESUCCESS", "message": "Comment updated successfully" }</pre>	<pre>{ "code": UPDTFAILS", "message":Attach ment doesn't exist exist" }</pre>

Name	Http Method	Description	Success Response	Error Response
<u>/api/attachments/delete/{attachmentId}</u>	DELETE	Delete an attachment.	<pre>{ "code": "DELETESUCCESS", "message": "Comment deleted successfully" }</pre>	<pre>{ "code": "DLTFAILS", "message": "Attacment doesn't exist" }</pre>
<u>/api/notifications/post</u>	POST	Create a new notification.	<pre>{ "code": "POSTSUCCESS", "message": "Notification added successfully" }</pre>	<pre>{ "code": "ADDFAILS", "message": "Notification already exist" }</pre>
<u>/api/notifications/all</u>	GET	Get a list of all notifications.	{List of object of Notification}	<pre>{ "code": "GETALLFAILS", "message": "Attachment list is empty" }</pre>
<u>/api/notifications/{notificationId}</u>	GET	Get details of a specific notification by ID.	{Object of Notification}	<pre>{ "code": "GETFAILS", "message": "Attachment doesn't exist" }</pre>
<u>/api/notifications/update/{notificationId}</u>	PUT	Update notification details.	<pre>{ "code": "UPDATESUCCESS", "message": "Notification updated successfully" }</pre>	<pre>{ "code": "UPDTFAILS", "message": "Notification doesn't exist" }</pre>
<u>/api/notifications/delete/{notificationId}</u>	DELETE	Delete a notification.	<pre>{ "code": "DELETESUCCESS", "message": "Notification" }</pre>	<pre>{ "code": "DLTFAILS", "message": "Notification" }</pre>

Name	Http Method	Description	Success Response	Error Response
			<code>deleted successfully"</code>	<code>tion doesn't exist"</code>
/api/userroles/post	POST	Create a new user role.	<code>{ "code": "POSTSUCCESS", "message": "UserRoles added successfully" }</code>	<code>{ "code": "ADDFAILS", "message": "UserRoles already exist" }</code>
/api/userroles/all	GET	Get a list of all user roles.	{List of object of UserRoles}	<code>{ "code": "GETALLFAILS", "message": "UserRoles list is empty" }</code>
/api/userroles/{roleId}	GET	Get details of a specific user role by ID.	{Object of userRoles}	<code>{ "code": "GETFAILS", "message": "UserRoles doesn't exist" }</code>
/api/userroles/update/{roleId}	PUT	Update user role details.	<code>{ "code": "UPDATESUCCESS", "message": "UserRoles updated successfully" }</code>	<code>{ "code": "UPDTFAILS", "message": "UserRoles doesn't exist exist" }</code>
/api/userroles/delete/{roleId}	DELETE	Delete a user role.	<code>{ "code": "DELETESUCCESS", "message": "UserRoles deleted successfully" }</code>	<code>{ "code": "DLTFAILS", "message": "UserRoles doesn't exist" }</code>
/api/userroles/assign/assign	POST	Assign a user role to a user.	<code>{ "code": "POSTSUCCESS", "message": "</code>	<code>{ "code": "ADDFAILS",</code>

Aa Name	⚙ Http Method	≡ Description	≡ Success Response	≡ Error Response
			<pre>"UserRole added successfully" }</pre>	<pre>"message":UserRole already exist" }</pre>
/api/userroles/all	GET	Get a list of all user-role associations.	{List of object of UserRole}	<pre>{ "code": GETALLFAILS", "message":UserRole list is empty" }</pre>
/api/userroles/user/{userId}	GET	Get user roles for a specific user.	{Object of UserRole}	<pre>{ "code": GETFAILS", "message":UserRole doesn't exist" }</pre>
/api/userroles/revoke/{userId}/{roleId}	DELETE	Revoke a user role from a user.	<pre>{ "code": "DELETESUCCESS", "message": "UserRole deleted successfully" }</pre>	<pre>{ "code": DLTFAILS", "message":UserRole doesn't exist" }</pre>